1) Identify if the given expressions are balanced or not

i) [a*(k+8-{6+e}+77)/100, ii) (((a*b)+[u/2]-d)

Ans

i) [a*(k+8-{6+e}+77/100

Symbol	Action	Stack
[	Push	[
(	Push	[(
{	Push	[({
}	Pop, matches {	[(
)	Pop, matches (	[
end of String	[still on stack	--

* Result:- It is unbalanced

ii) (((a*b)+[u/2]-d)

Symbol	Action	Stack
(	Push	(
(	Push	((
(	Push	(((
)	Pop	((
[	Push	(((
]	Pop	((
)	Pop	(
end of string	(still on stack	-

Result:- It is unbalanced.

2) Given a stack with repeated elements, design an approach to compress it (value & frequency) while maintaining stack operations. Apply stack ADT modifications. Analyze trade-off between time and space.

Ans

*A Stack is a linear data structure that follows the Last In, First out (LIFO) Principle.

*In some applications, the stack may contain many consecutive repeated elements.

For example, instead of storing:

5,5,5,4,4,2,2,2

the compressed stack stores:

(5,3),(4,2),(2,3)

Modified stack ADT:

*each node in the stack contains 3 fields:

*Value - stores the element

* Frequency - stores the number of consecutive occurrences.

* Next - points to the next node in stack

This modification allows the stack to store repeated elements efficiently without changing its LIFO behaviour.

Time complexity:

Operation	Time complexity
Push	o(1)
Pop	o(1)
Peek	o(1)
Compression	o(n)

Space complexity:

*original stack : o(n)

*compressed stack : o(k)

Where K is the number of distinct consecutive groups and $K \leq n$.

eg: original :- 5555555 , compressed : (5,4).

Space = 4 nodes, Space node = 1.

Trade off between time and space.

Aspect	original stack	compressed stack
memory usage	High	Low
Push	o(1)	o(1)
Pop	o(1)	o(1)
Implementation	simple	slightly complex
Best case	No duplicates	Many duplicates
Space efficiency	Poor	Excellent.

Push operation:

When a new element is pushed:

1) If the stack is empty, create a new node with frequency equal to 1.

2) If the new element is the same as the value at the top of the stack, increase the frequency by 1.

Eg: Push sequence : 5, 5, 5, 4, 4, 2

Compressed Stack: Top
(2, 1)
(4, 2)
(5, 3)
Bottom

Pop operation:

When an element is poped:

1) If the frequency of the top node is greater than 1, decrease the frequency by 1.

2) If it becomes 1, remove the entire node from the stack.

Peek operation:

*The peek operation returns the value stored in the top node without removing it. The frequency remains unchanged.